驾驶agent-draft-1009

汇报

Multi-Agent 自动驾驶应用：（两条技术路线）

车辆间协同：将不同车辆建模为独立 Agent，重点研究多车之间的交互、博弈、通信与协同决策

典型应用：匝道汇入、交叉口、环岛和混合交通流

代表工作：KoMA、AgentsCoDriver 和 CoMAL

机制：LLM 根据其他车辆状态进行意图推断、通过 Shared Memory 共享历史驾驶经验，以及利用自然语言或 Shared Message Pool 显式交换当前意图和协同策略。

发展趋势：独立决策→其他车辆意图推断→共享记忆→显式自然语言通信→多车策略协商和角色分配

车内功能协同：单车内部不同功能 Agent 之间的任务分工与协作（将感知、推理、规划和安全检查拆分为多个 Agent，再通过协作完成最终决策）。该方向的思想是将传统端到端规划转变为“任务分解—专业Agent执行—结果聚合与修正”的协作式多Agent框架。

代表工作：DriveAgent 将多模态感知、车辆状态分析、环境推理和响应生成划分为不同功能 Agent；DAPlanner 采用双 Agent 结构，分别负责轨迹生成和异常检测与修正。

Agent-Driver：单Agent + Tool + Memory + Reasoning + Reflection 架构。本身并非严格意义上的 Multi-Agent，而是由单一 LLM 统一调用感知、预测、地图等工具，并结合记忆、推理和反思完成规划，是传统单Agent 向功能型 Multi-Agent 发展的中间形态。

draft

多智能体问题建模：相互影响与联合决策

（Complex Task - Subtasks - Specialized Agents - Aggregation）

其中，单个 Agent 通常包含：

感知（Perception）：获取环境信息

推理（Reasoning）：分析当前状态

规划（Planning）：制定行动方案

执行（Action）：调用工具或产生输出

记忆（Memory）：保存历史信息

智能驾驶 多次调用 LLM 处理不同任务

范式 Complex Task - Subtasks - Specialized Agents - Aggregation

两个方向？多车协同，单车功能协同

A Language Agent for Autonomous Driving（Agent-Driver）

系统概括：**Tool Library + Cognitive Memory + Reasoning Engine**，用LLM统一调度

1、背景/动机

自动驾驶与人类驾驶员相比，缺少长期驾驶经验和先验知识

传统“感知-预测-规划”系统的问题：

①感知信息冗余

人类往往只关注少量关键目标，而传统系统需要处理较大感知范围内的大量目标

“是否可以让一个智能体主动判断“我现在需要什么信息”，而不是把全部感知结果都塞给规划器？”

②缺少常识和经验

预测和规划是仅针对检测到的物体，缺乏深层推理能力

Human Knowledge=Language 语言方式

Driving Neural Networks=Numerical Features （eg.感知信号、边界框）

这种**语言和数值之间的表征差异，**限制神经网络利用先验知识来提高驾驶性能

③缺少显示推理

解决思路：LLM作为认知代理（系统之间的调度器），把自然语言作为统一的接口

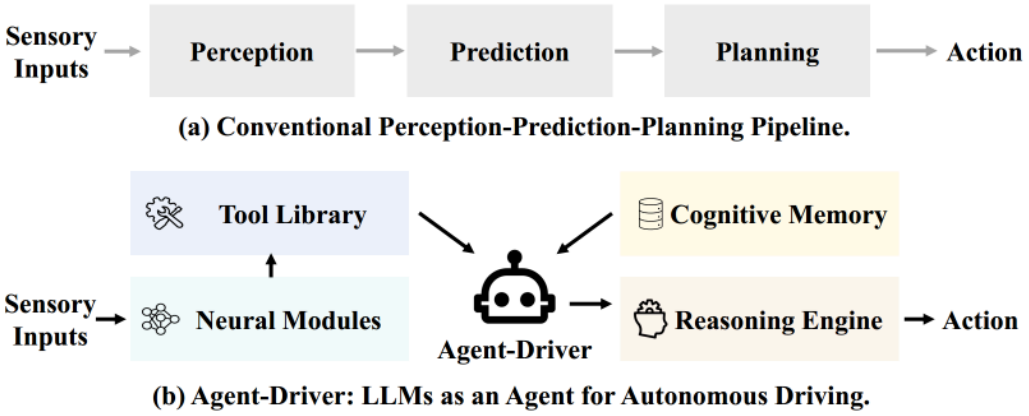


fig1系统对比

（1）Tool Library 工具库：动态函数调用，减少感知冗余

（只有当LLM决定调用相关函数时，神经模块才被激活）

（2）Cognitive Memory 认知存储器：存储常识和人类驾驶经验

（3）Rasoning Engine推理引擎：处理感知结果和存储器数据，模拟人类决策

关键对象/事件识别-CoT，任务规划-高级驾驶意图，运动规划-生成驾驶轨迹，自我反思-保证安全性

2、总体框架

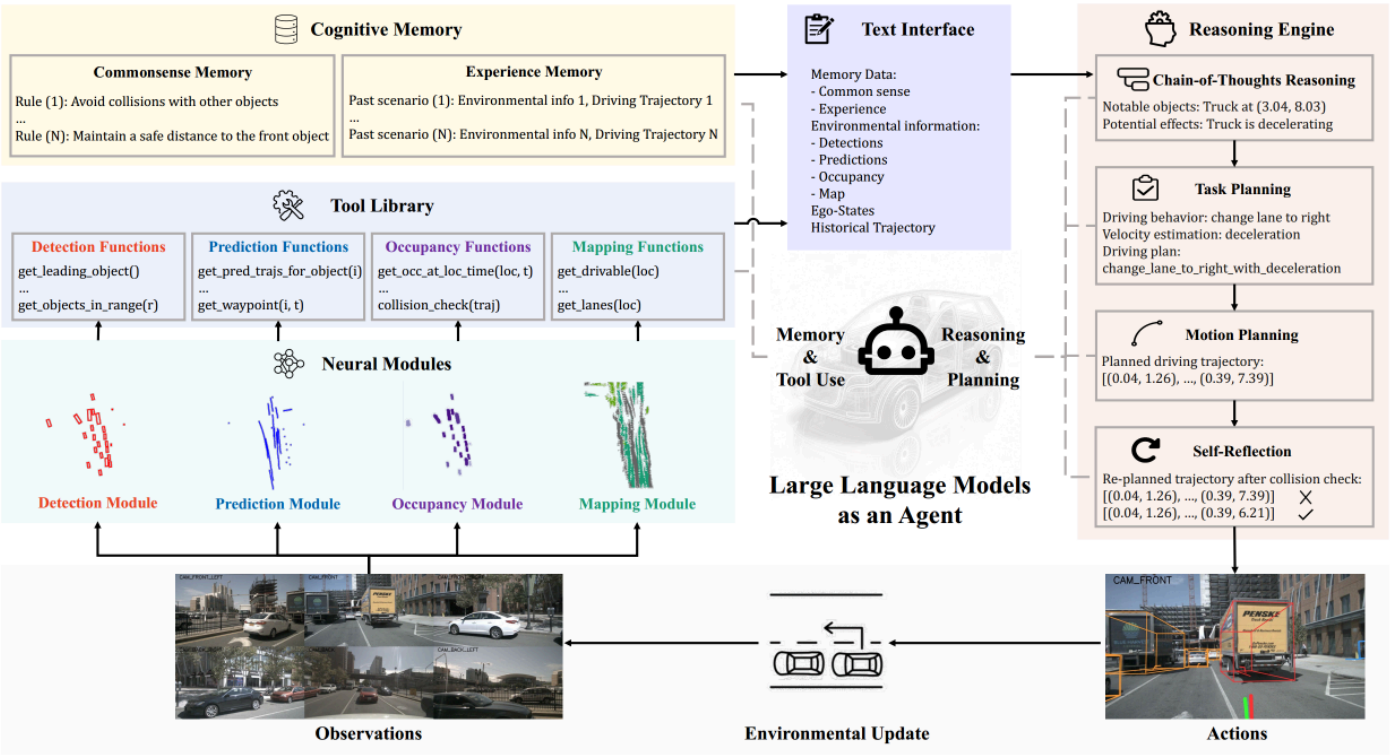


fig2总体框架

数据流：Sensor-Neural Modules-Tool Library-Memory-Reasoning-Planning-Reflection-Traj.

神经模块仍然负责 Detection、Prediction、Occupancy 和 Map，LLM 知识作为上层“认知层”，论文主要使用的是 **GPT-3.5-turbo-0613**，不是视觉语言模型，视觉等传感器信息先由已有的感知神经网络处理。

3、方法

（1）Tool library：动态收集文本形式的环境信息

传统方案：直接把感知网络的输出给规划器；将先验信息转换成语义特征

Agent-Driver：**把感知模型封装成一系列函数（20+）**；将神经网络输出汇总为基于文本的信息

Functions：附录2.1 table6、7（检测、预测、占用、地图）

类别	作用	示例	
Detection	查询目标	get_leading_object()	返回对象id、位置、大小
Prediction	查询目标未来运动	get_pred_trajs_for_object(i)	返回路点轨迹
Occupancy	查询某位置是否被占用	get_occ_at_loc_time(loc,t)	返回占用概率
Map	查询道路结构	get_lanes()	返回左右车道信息

Tool Use： dynamic function calls 只查询当前决策真正需要的信息。

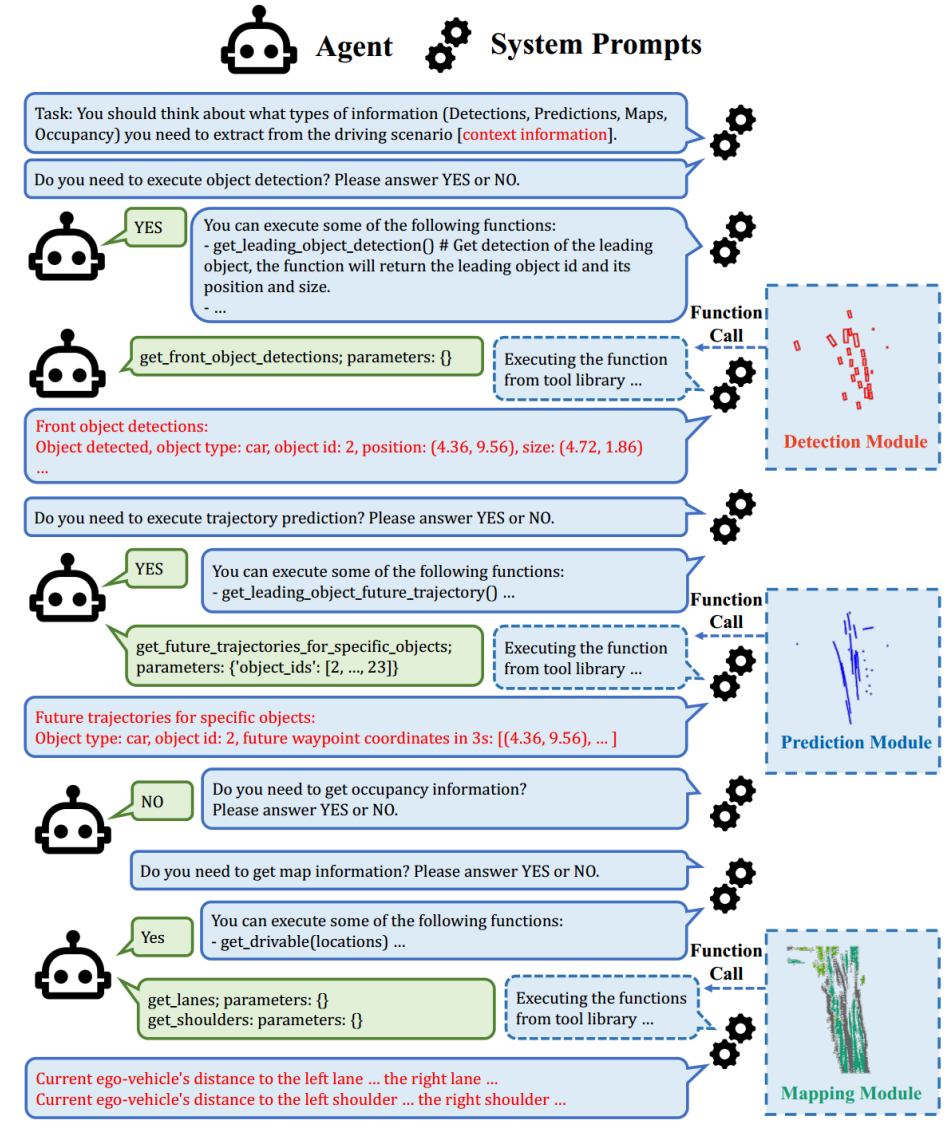


fig3.工具库函数调用示例（多轮对话）（示例内容 附录fig4、5）

（2）Cognitive Memory： 利用收集到的环境信息作为查询，在存储器中搜索相似的过去经验来辅助驾驶

文本形式 Memory=Mcommonsense+Mexperience

常识存储器：（交通规则、危险行为…）可编辑的文本，直接写入（不是神经网络权重，无需重新训练模型）

经验存储器：（过去驾驶场景，包含环境信息和驾驶决策）类似：Retrieval-Augmented Memory

记忆检索过程：**两阶段检索算法** Vector Search+LLM Fuzzy Search （附录B）

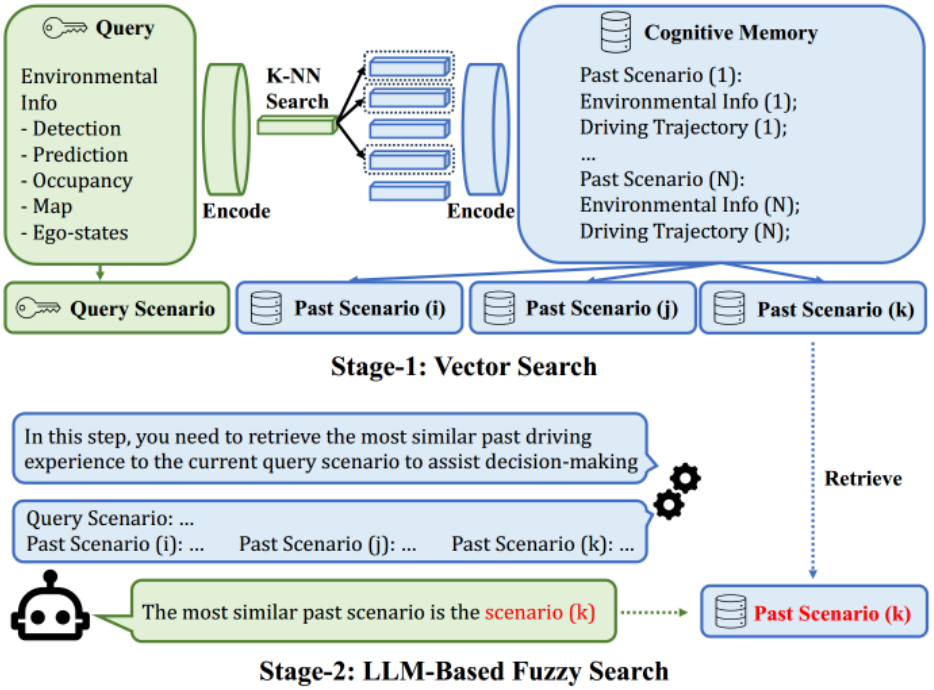


fig4：两阶段记忆检索 （示例内容 附录fig6-8）

Stage-1：通过KNN找到找到 Top-K 候选

构造过往场景向量： $k_i = [e_i, g_i, h_i]$ ego states + mission goal + historical traj. 与当前场景 $Q = [e, g, h]$

计算加权相似度： $S = Q\Lambda K^T$ ，其中 $\Lambda = \mathrm{diag}(\lambda_e, \lambda_g, \lambda_h)$ （权重）

Stage-2：LLM fuzzy matching 利用LLM理解候选场景的文本描述，选出最相似的一个。

（3）Reasoning Engine（附录C）：思维链推理、任务规划、动作规划和自我反思

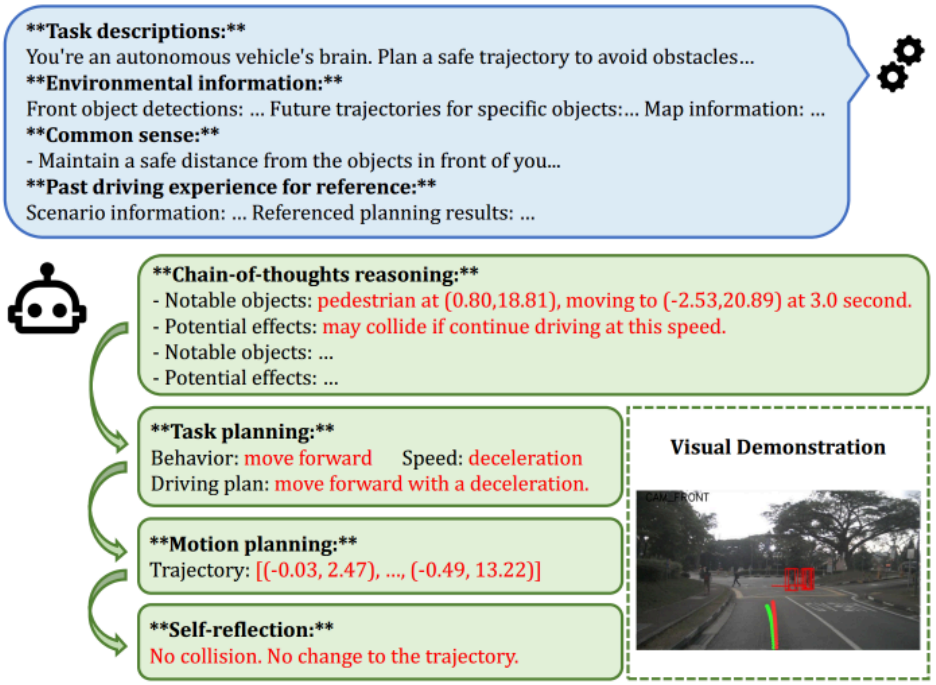


fig5：reasoning engine （示例内容 附录fig9-11）

CoT：关键性判别（Notable objects；Potential effects）

策略：上下文学习（few-shot）/ 微调（GPT-Driver）

Table 5 ： 上下文学习使LLM能够生成更多样化的推理输出，并产生更好的运动规划性能

Task Planning：不直接执行运动规划（传统），在envir.和traj.中间增加高层驾驶意图，表示自车的大致运动

离散驾驶行为（6种）和速度（6种）的组合（31种），eg.change_lane_to_left_with_deceleration

策略：上下文学习 / 微调

Driving behavior	Speed estimation
move_forward	constant_speed
change_lane_to_left	deceleration
change_lane_to_right	quick_deceleration
turn_left	deceleration_to_zero
turn_right	acceleration
stop	quick_acceleration

Table 1: Driving behaviors and speed estimations.

Motion Planning：将运动规划转换为语言建模（GPT-Driver）

$\tau = F_{\text{LLM}}(O, M, R, P)$ ， $\tau = [(x_1, y_1), (x_2, y_2), \ldots, (x_6, y_6)]$

基于驾驶计划P、推理结果R、环境信息O和记忆数据M，生成未来 3 秒中的 6 个路点

Self-Reflection：碰撞检查和优化

对于来自运动规划模块的规划轨迹τ，调用工具库中的collection_check(traj)函数来检查其碰撞

如果检测到碰撞，通过优化成本函数C将轨迹τ优化为新轨迹：

$\tau^* = \min_{\tau} C(\tau, \hat{\tau}) = \min_{\tau} \lambda_1 \|\tau - \hat{\tau}\|_2 + \lambda_2 \mathcal{F}_{\text{col}}(\tau)$，前一半约束新轨迹和原来规划轨迹的距离，后一半尽量远离碰撞区域（附录C.4）

4、实验

模型：使用 **gpt-3.5-turbo-0613** 作为Agent-Driver中的基础LLM；

只有运动规划，单独使用经过轨迹微调的 LLM

4.1 开环：nuScenes（完整训练数据）

	Method	L2 (m) ↓				Collision (%) ↓			
		1s	2s	3s	Avg.	1s	2s	3s	Avg.
ST-P3 metrics	ST-P3 (Hu et al., 2022)	1.33	2.11	2.90	2.11	0.23	0.62	1.27	0.71
	VAD (Jiang et al., 2023)	0.17	0.34	**0.60**	0.37	0.07	0.10	0.24	0.14
	GPT-Driver (Mao et al., 2023a)	0.20	0.40	0.70	0.44	0.04	0.12	0.36	0.17
	Agent-Driver (ours)	**0.16**	**0.34**	0.61	**0.37**	**0.02**	**0.07**	**0.18**	**0.09**
UniAD metrics	NMP (Zeng et al., 2019)	-	-	2.31	-	-	-	1.92	-
	SA-NMP (Zeng et al., 2019)	-	-	2.05	-	-	-	1.59	-
	FF (Hu et al., 2021)	0.55	1.20	2.54	1.43	0.06	0.17	1.07	0.43
	EO (Khurana et al., 2022)	0.67	1.36	2.78	1.60	0.04	0.09	0.88	0.33
	UniAD (Hu et al., 2023)	0.48	0.96	1.65	1.03	0.05	0.17	0.71	0.31
	GPT-Driver (Mao et al., 2023a)	0.27	0.74	1.52	0.84	0.07	0.15	1.10	0.44
	Agent-Driver (ours)	**0.22**	**0.65**	**1.34**	**0.74**	**0.02**	**0.13**	**0.48**	**0.21**

Table2：开环表现——主要体现在超过 30% 的 collision improvement，决策安全性提升。
两种指标主要区别：是否做时间平均

附录D3：消融实验（10%训练数据）

ID	Tool Library	Common. Memory	Exp. Memory	CoT Reason.	Task Plan.	Self-Reflect.	L2 (m) ↓				Collision (%) ↓			
							1s	2s	3s	Avg.	1s	2s	3s	Avg.
1	✗	✓	✓	✓	✓	✗	0.24	0.71	1.44	0.80	0.03	0.27	0.91	0.40
2	✓	✗	✓	✓	✓	✗	0.24	0.69	1.42	0.79	0.03	0.23	0.83	0.37
3	✓	✓	✗	✓	✓	✗	0.24	0.72	1.46	0.81	0.07	0.23	0.96	0.42
4	✓	✓	✓	✗	✓	✗	0.24	0.71	1.45	0.80	0.03	0.23	0.88	0.38
5	✓	✓	✓	✓	✗	✗	0.25	0.72	1.47	0.81	0.05	0.23	0.93	0.40
6	✓	✓	✓	✓	✓	✗	0.24	0.70	1.42	0.79	0.03	0.20	0.81	0.35
7	✓	✓	✓	✓	✓	✓	0.25	0.71	1.43	0.80	0.03	0.08	0.56	0.23

附录Table2：去掉 Tool Library 后，所有感知结果直接进入 LLM，输入 token 数大约变成原来的 2 倍，collision 也会上升；②Self-Reflection 对 Collision 的作用明显（主要改善安全性，而非imitation accuracy）

4.2 闭环：CARLA （Town05-Short benchmark）

Town05-Short 包含10条挑战路线，每条路线包含 3 个 intersection，并有大量动态交通参与者

Methods	Driving Score ↑	Route Completion ↑
CILRS (Codevilla et al., 2019)	7.47	13.40
LBC (Cui et al., 2021)	30.97	55.01
Transfuser (Prakash et al., 2021)	54.52	78.41
ST-P3 (Hu et al., 2022)	55.14	86.74
VAD (Jiang et al., 2023)	**64.29**	87.26
Agent-Driver (Ours)	57.33	**91.37**

Table3：闭环表现——Driving Socre持平，Route Completion提高4.1%

4.3 Few-shot learning 实验：验证泛化性



fig6：few-shot实验结果——即使只有 0.1%（23 条样本），仍能得到可用结果；使用 1% 数据以后，Agent-Driver 已经能够超过用完整数据训练的 UniAD baseline，尤其是 collision 指标。

Motion-Planning LLM 分别只使用：0.1%（23条样本）,1%,10%,50%,100%的训练数据进行微调

4.4 可解释性

4.5 兼容性

Method	L2 (m) ↓				Collision (%) ↓			
	1s	2s	3s	Avg.	1s	2s	3s	Avg.
Llama-2-7B	0.25	0.69	1.47	0.80	0.02	0.27	0.78	0.35
gpt-3.5-turbo-1106	0.24	0.71	1.47	0.80	0.03	0.08	0.63	0.25
gpt-3.5-turbo-0613	0.22	0.65	1.34	0.74	0.02	0.13	0.48	0.21

Table 3: **Compatibility to different LLMs.**

附录D5 与其他神经网络模块的兼容性

附录D6 Tool library 具有拓展性，通过外接vlm，可实现视觉语言解释能力

4.6 稳定性（仅验证输出格式的稳定性）

4.7 In-context Learning / Fine-tuning

Modules		Avg. L2 (m)	Avg. Col. (%)
CoT Reason.+Task Plan.	Motion Plan.		
Fine-tuning	In-context learning	1.81	0.79
In-context learning	In-context learning	1.90	0.79
Fine-tuning	Fine-tuning	0.72	0.22
In-context learning	Fine-tuning	0.74	0.21

Table 5: **In-context learning vs. fine-tuning.** In-context learning performs slightly better in reasoning and task planning. Fine-tuning is indispensable for motion planning.

Reasoning / Task Planning - ICL ; Motion Planning 必须 Fine-tune

5、总结

1. **贡献**

   提出 Agent-Driver：让 LLM 从单一规划器变为自动驾驶上层 Agent（调度角色）。

   构建 Tool Library，动态调用检测、预测、Occupancy、Map 等模块，减少无关信息。

   显式 Cognitive Memory，包含驾驶常识和历史经验。

   将决策拆成 CoT Reasoning → Task Planning → Motion Planning，提升可解释性。

   增加 Self-Reflection，通过碰撞检测和轨迹优化降低碰撞率。

2. **局限**

   LLM 多轮调用带来较大计算和时延，实时性尚未验证。

   依赖上游感知、预测结果，错误的航向角（失败案例）等会直接影响规划。模型解决的是“如何基于高质量结构化信息做智能决策”，没有解决“如何直接从复杂视觉世界获得可靠语义”。（在D.6额外讨论把VLM当作tool，增加视觉语言解释能力。）

   Tool library 依赖人工定义函数

   （未来可研究更高效的小模型、动态工具选择、更强 Memory，以及更全面的 reasoning/planning reflection）

3. **与我的工作的关联 / 启发**

   Agent-Driver 的 Detection Tool 仍然受传统 detector 类别集合限制。VLM 可补充传统检测器对：沙堆、草地、异常障碍物等开放类别的识别能力的欠缺，从原始视觉中发现、定位规划关键目标。

   关键目标识别：Notable object + Potential effect

   Self-Reflection，在规划后增加碰撞/安全检查，形成“感知—推理—规划—验证”闭环。

DriveAgent: Multi-Agent Structured Reasoning With LLM and Multimodal Sensor Fusion for Autonomous Driving

DriveAgent **只使用一个微调后的 LLaMA-3.2-Vision-11B VLM（接收不同输入，执行不同 prompt）**；它不是多个 LLM/VLM 并行，而是通过多个功能 Agent 进行任务分工（任务分解）。只有 Visual-based Object Description 阶段直接利用 VLM 对图像进行视觉理解；LiDAR-based Object Description、Vehicle Reasoning、Environmental Reasoning 和 Response Generation 主要处理结构化传感器信息或前序文本描述，因此主要依赖 LLM 的语言理解与推理能力。

1、背景/动机

复杂动态场景中：跨模态信息无法有效整合、视觉信息缺失（或信息冲突异常）时鲁棒性不足、决策过程可解释性差等问题。

现有 LLM 自动驾驶方法多集中于单一模态、端到端规划或简单 CoT 推理，对 Camera、LiDAR、IMU、GPS 之间的协同分析和时序因果推理仍不充分。因此，作者提出 **DriveAgent**，通过**多个专门 Agent 分工处理不同模态信息，并利用 LLM 进行跨模态融合、环境变化分析和因果推理**，以提升复杂驾驶场景下的鲁棒性、可靠性和可解释性。

现有自动驾驶 AI Agent 虽然能够分别处理视觉、语言和决策任务，但这些能力往往彼此割裂，缺乏真正统一的多模态推理机制。

2、总体框架

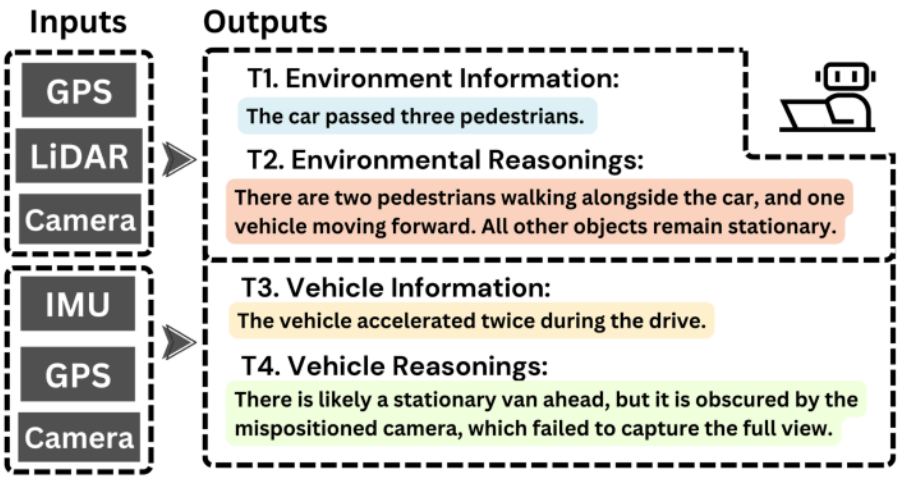


fig1. DriveAgent输入输出，数据处理的四个任务

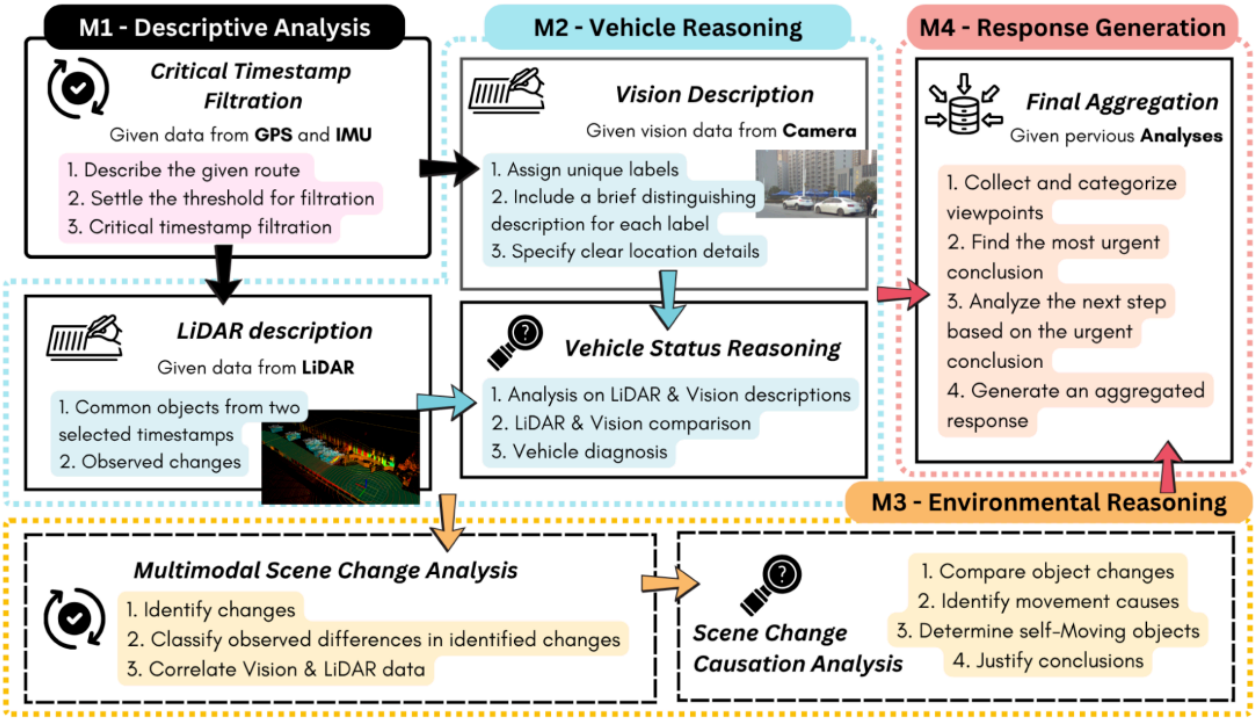


fig2.整体架构——四个模块（M1至M4），其中多模态传感器输入相机，IMU，GPS和激光雷达，最终实现环境级任务（例如，信息检索，环境变化检测和推理）和车辆级任务（例如，车辆状态分析，运动评估和行为模式识别）

DriveAgent 被拆成四个模块：M1 : Descriptive Analysis、M2 : Vehicle Reasoning、M3 : Environmental Reasoning、M4 : Response Generation

找关键时刻-判断车辆和传感器状态-理解环境变化和原因-生成最终建议

3、方法

A. Module1 —— Descriptive Analysis

驾驶数据庞大且连续帧信息重复 → LLM 先根据阈值筛选 关键时间点（车辆运动状态显著变化）：$\{(T_i, F_i)\}_{i=0}^{n}$ ，（time/reason）。

不同路线筛选阈值不同：依据车辆速度 S 和城市复杂度指标 U ，把当前驾驶路线划分到某一类 r_i ，并给这个类别对应一个筛选阈值 θ_i 。基线：角速度：$10^\circ/s$ 、线性加速度：$8m/s^2$ 、偏航率：$10^\circ/s$ 。

IMU 提供 角速度、加速度、偏航率

GPS 提供全局运动参考 和 多传感器时间同步

B. Module 2 ——Vehicle Reasoning

包含 Vision Agent + LiDAR Agent + Analyzer Agent

① Vision Agent：计算特定目标在图像或空间描述里发生的位移

为相机中检测到的对象分配索引ID，在时间t：$p_i(t)$ 及下一时刻 $p_i(t+1)$ $p_i(t) \sim p_i(t+1)$ （跨帧关联），从而计算位移：$\Delta p_i = p_i(t+1) - p_i(t)$ 。

② LiDAR Agent：基于点云计算物体位移，类似 ①

③ Vehicle Status Reasoning：判断 Camera 和 LiDAR 结果是否一致

如果只有一个物体异常，可能是这个物体识别错误；如果大量物体同时 Camera–LiDAR 差异变大，更可能是整个传感器发生了问题。

C. Module 3 —— Environmental Reasoning

短时环境变化：对比视觉检测变化和LiDAR检测变化，识别 new, missing, or significantly relocated objects。

原因分析：分析一个物体较长时间的状态变化，并判断是 self-moving 还是 externally influenced

“ 持续性不一致，可能说明传感器存在问题。一致性的位移变化，可能说明是真实的环境变化。”

D. Module 4: Response Generation

通过 urgency scoring 找到最紧急的问题，针对该问题从候选响应中选出最终响应。

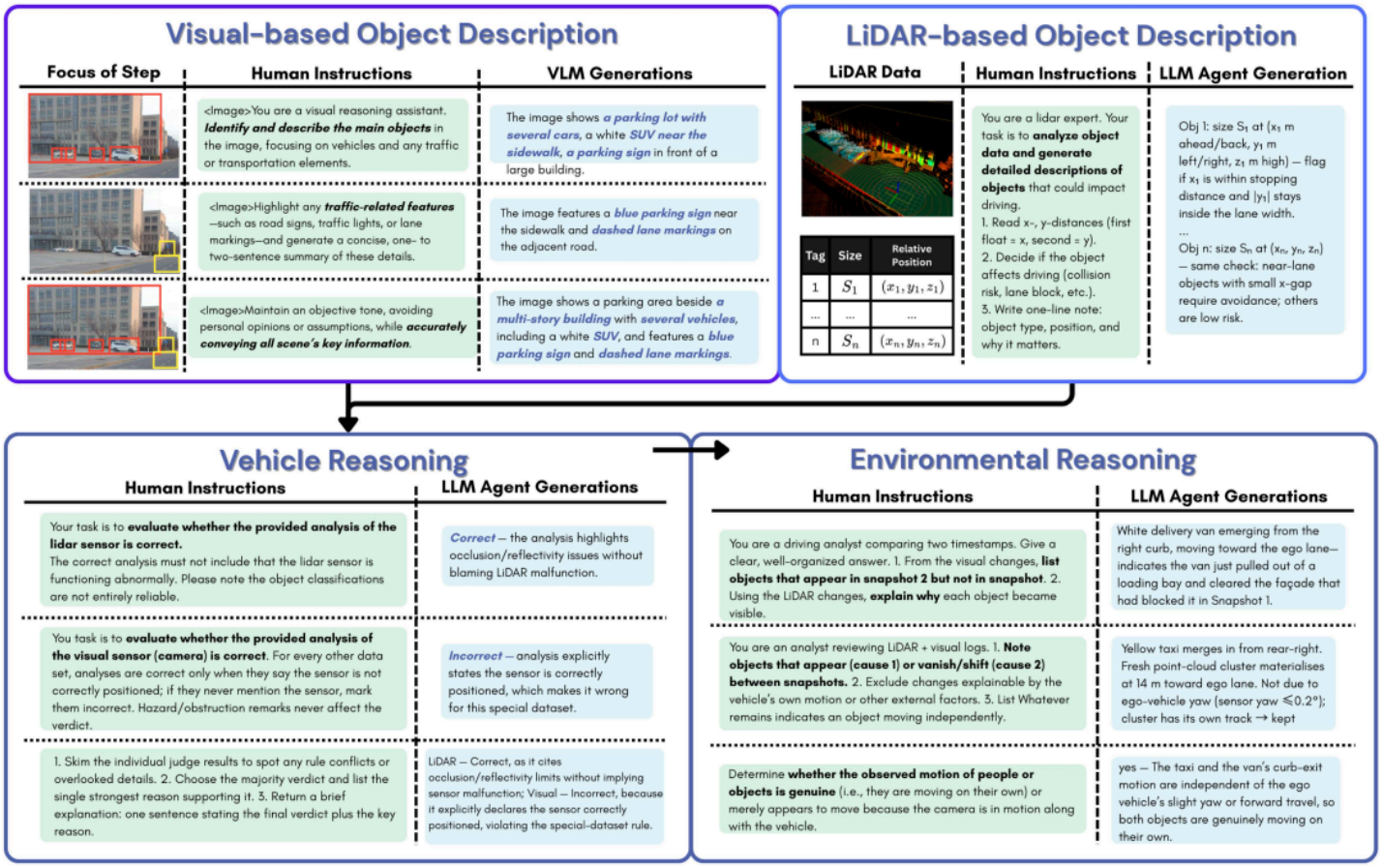


fig4.多智能体、多模态的驾驶异常分析pipeline

1. **Visual-based Object Description**

   输入：相机图像；通过人工提示让 VLM 描述场景中的主要目标、交通元素、道路标志等；输出：视觉场景描述

2. **LiDAR-based Object Description**

   输入：LiDAR 点云及其结构化目标信息，如尺寸 (S_i)、相对位置 (x_i,y_i,z_i)

   LLM Agent 根据目标大小、位置、车道关系等判断哪些目标可能影响驾驶

   输出：LiDAR 目标描述与风险信息

3. **Vehicle Reasoning**

   分别检查：LiDAR 和 Camera 分析是否合理，最终得到一个关于**车辆传感器状态/异常情况**的判断

4. **Environmental Reasoning**

   比较两个不同时间戳的 Camera 和 LiDAR 描述，判断：哪些目标出现/消失，位置变化是自车运动造成的相对变化/确实在独立运动

   先让 Camera 和 LiDAR 分别形成语言化描述，再通过 LLM Agent 对两种模态进行交叉验证，并进一步区分“传感器问题”和“真实环境变化”。

4、实验

数据集：使用真实自动驾驶车辆采集数据（长安大学三条路线Fig3、Table2），R1基础，R2动态复杂，R3结构复杂。传感器包含 Camera、LiDAR、GPS/GNSS、IMU，并进行了多传感器时间同步（Table1）。

任务与评估：

任务1——目标与类别检测，7个类别，用R2、R3训练，R1测试，指标： Precision, Recall, F1。

任务2——车辆推理，判断 Camera/LiDAR 数据是否异常

任务3——环境推理，判断哪些目标是 independent moving object，而非车辆相对运动产生的位移变化（树）

后两个任务的评估集中在两个方面：车辆诊断推理的准确性，以及环境和因果推理的准确性。

baseline：任务1对比5个VLM，任务23比较三种推理方法：Zero-shot、CoT、CoT+Self-Refine

VLM模型：LLaMA-3.2-Vision 11B

训练：vision encoder：LoRA；language model：LoRA；projection layer：全参数。

训练配置：NVIDIA H100；Optimizer——AdamW；

训练数据使用结构化 JSON instruction-response pairs，并使用 `<Image>` token 表示视觉输入，进行 supervised instruction tuning

结果分析：

任务1. 目标与类别检测 （R1）

Model	Precision	Recall	F1-score
LLaMA-3.2-Vision-Instruct	64.33	35.26	45.55
GPT-4o-mini	64.40	51.60	57.30
pixtra-large	76.12	54.63	60.86
GPT-4o	80.98	59.98	64.96
claude-3.7-sonnet	83.83	69.73	68.80
DriveAgent	89.96	63.52	71.62

Table3.微调后的 VLM 在自己的结构化类别定义和标注体系下学习效果显著

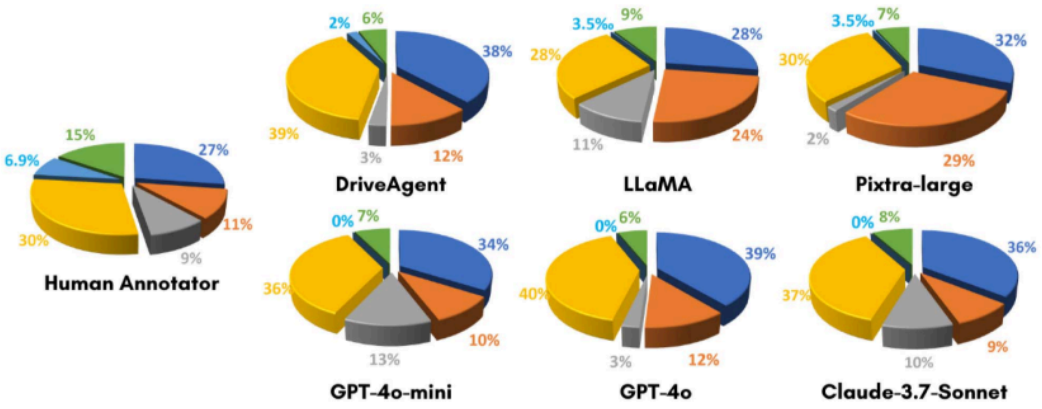


fig5.DriveAgent 预测出来的目标类别分布，比通用 VLM 更接近人工标注，尤其是在一些“自动驾驶领域特有类别”上：（浅蓝色）Monitor——道路上方电子信息显示屏、信息板、交通监控/显示设施等

任务23.推理表现

Tasks	Method	R2	R2-left	R2-right	R3	R3-left	R3-right
Vehicle Understanding (Lidar)	Zero-Shot	62.11	65.05	52.78	57.14	47.50	55.26
	CoT	15.79	22.33	15.74	18.10	20.00	13.16
	CoT + Self Refine	72.63	66.02	63.89	64.76	45.00	55.26
	DriveAgent (ours)	65.26	69.90	58.33	55.24	51.25	50.00
Vehicle Reasoning (Vision)	Zero-Shot	70.53	2.91	0.93	82.86	6.25	3.95
	CoT	69.47	25.24	27.78	79.05	31.25	40.79
	CoT + Self Refine	65.26	0.97	2.78	80.00	3.75	5.26
	DriveAgent (ours)	96.84	58.25	71.30	87.62	68.75	63.16
Environmental Reasoning	Zero-Shot	37.89	-	-	36.19	-	-
	CoT	56.84	-	-	62.86	-	-
	CoT + Self Refine	43.16	-	-	56.19	-	-
	DriveAgent (ours)	58.95	-	-	65.71	-	-
Ablation	DriveAgent (w/o causation)	56.28	-	-	63.77	-	-

Table4.分别比较不同模态、不同路线/相机视角下的推理准确率；R-left和R-right表示路线的左侧和右侧相机视图，是车辆推理子任务的干扰项（看能否发现不匹配）

（Vision）经过训练和结构化推理后，模型对 misaligned camera 识别效果更好

（环境推理）加入时序比较后，（比单纯增加通用语言推理）模型对环境变化判断效果更好

（因果分析）贡献不大 2%

论文提及的causal reasoning 更接近**基于时序证据的 LLM 因果解释**，而不是严格意义上的因果推断。

KoMA: Knowledge-driven Multi-agent Framework for Autonomous Driving with Large Language Models

决策框架，感知来自仿真环境文本描述；

Multi-Agent：两个 LLM 通过推断彼此行为意图实现交互（隐式），不涉及 LLM 参数训练

1、背景/动机

系统架构：

Data-driven：数据集偏差、过度拟合、可解释性差

Knowledge-driven：基于经验、知识及推理能力（适配 LLM 已有常识及推理能力）

Single agent 仅适用于简单、冲突较弱的场景中进行，无法满足 多车辆冲突场景（匝道汇入）

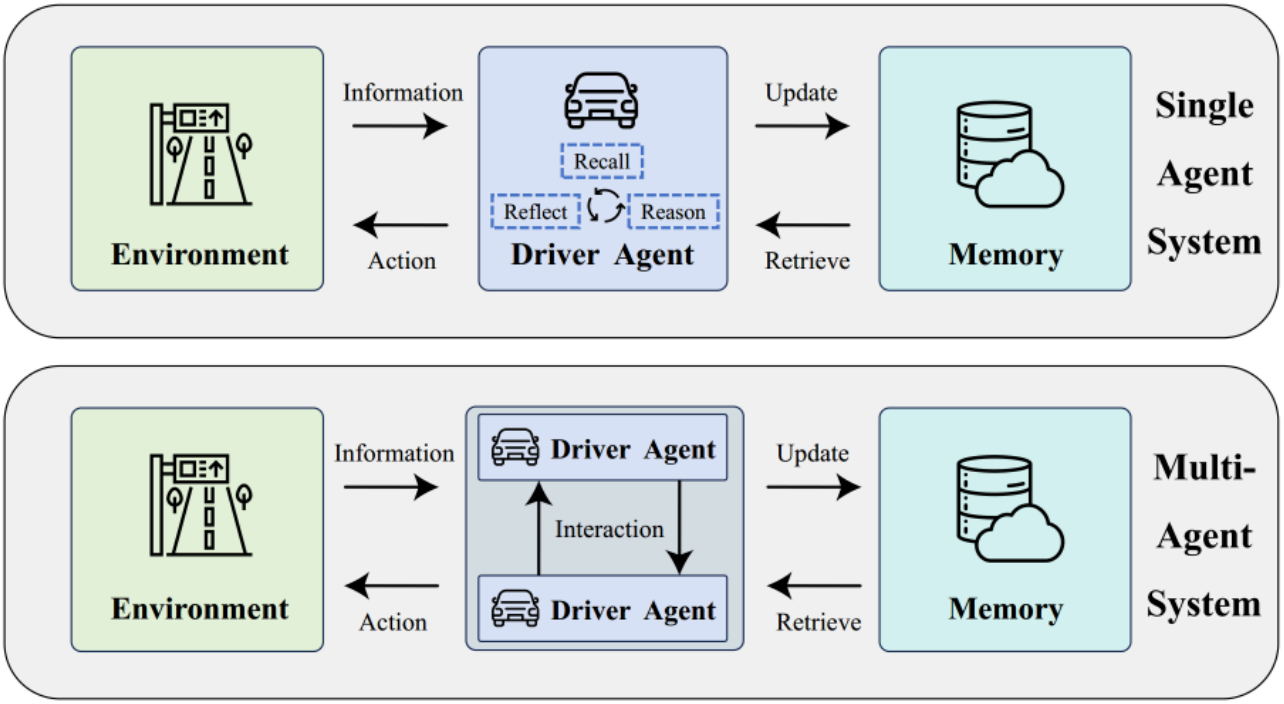


Fig1.单agent和多agent框架

推理：KoMA 结合单步决策的灵活和多步决策的一致性，提出 GOAL — PLAN — ACTION 结构 ，Plan 保持较长时间，但 Action 每个周期重新决定（Multi-step planning）

反馈：传统 Reflection 在发生碰撞后激活，而 KoMA 给每个 Action 计算 Safety Score、Efficiency Score，扩大Reflection 范围，在轨迹中找到真正引发后续碰撞的“低质量决策”

记忆：KoMA 提出 shared memory module （共享驾驶经验），允许多智能体共享一个向量数据库

2、总体框架

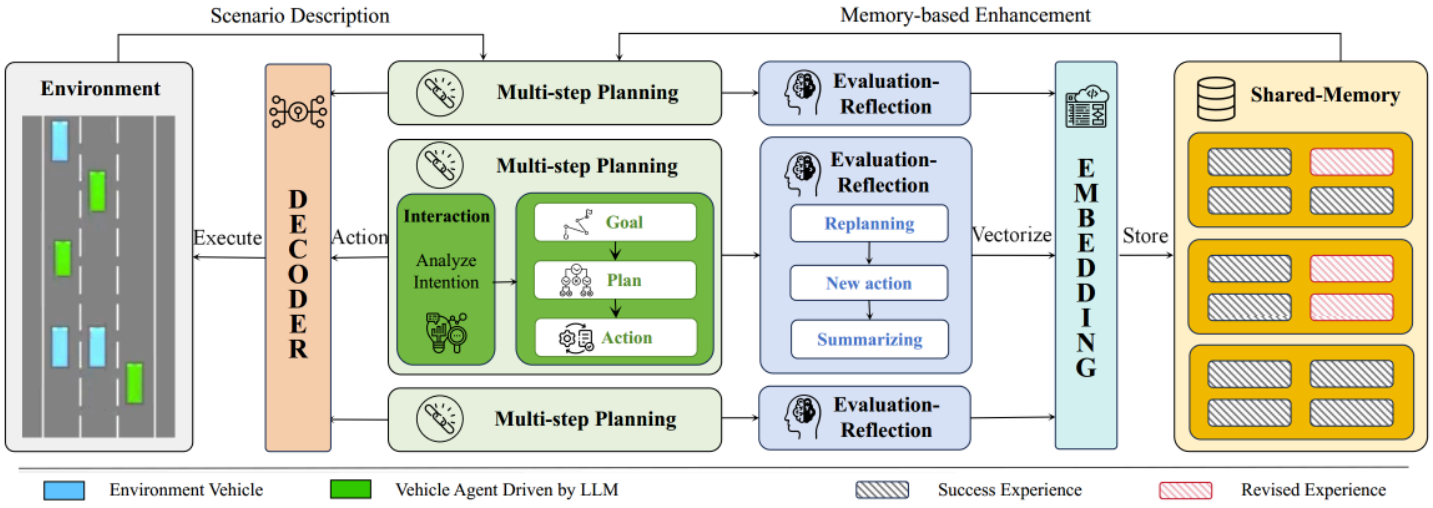


fig2.five core modules:
environment, interaction, multi-step planning, ranking-based reflection, shared memory module

3、方法

Environment：text description of scenario

Interaction：LLM 通过分析其他车辆的历史和当前状态，隐式推理意图

Planning：GPA 三层结构（绿色）

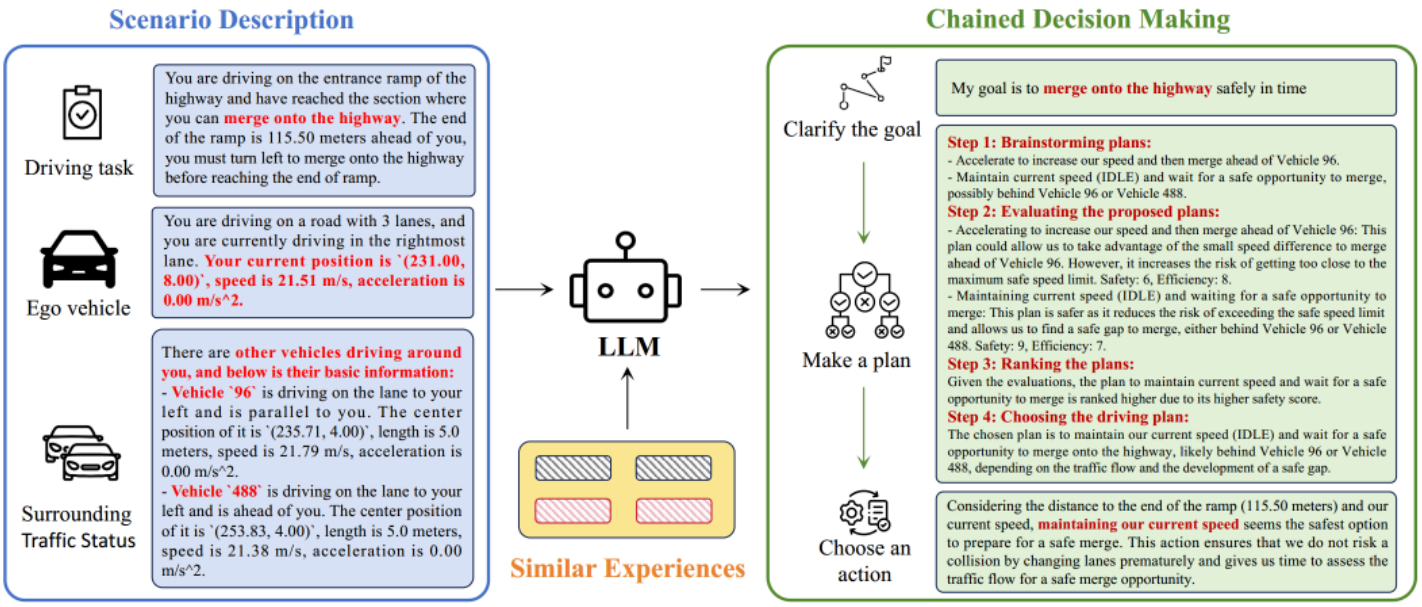


Fig.3 多步规划推理过程

Goal：LLM 基于场景明确目标（场景人为划分特殊和一般两类）

Plan（跨时间保持）：做规划会先检查上一时刻的计划可行性（基于其他车辆意图推理），可行则保留，不可行/没有计划则重新规划。论文并没有设计一个严格的数值优化函数，而主要由 **LLM 根据安全、效率进行语言推理后打分并排序。**

Action（逐步更新）：定义离散动作空间，
$\mathcal{A} = \{\text{Turn-left}, \text{IDLE}, \text{Turn-right}, \text{Accelerate}, \text{Decelerate}\}$

Shared Memory：RAG 思路

把场景文本描述转换为向量，最相似性对比

Experience 包含：Scenario Description、Planning Process、Final Decision、Evaluation Score

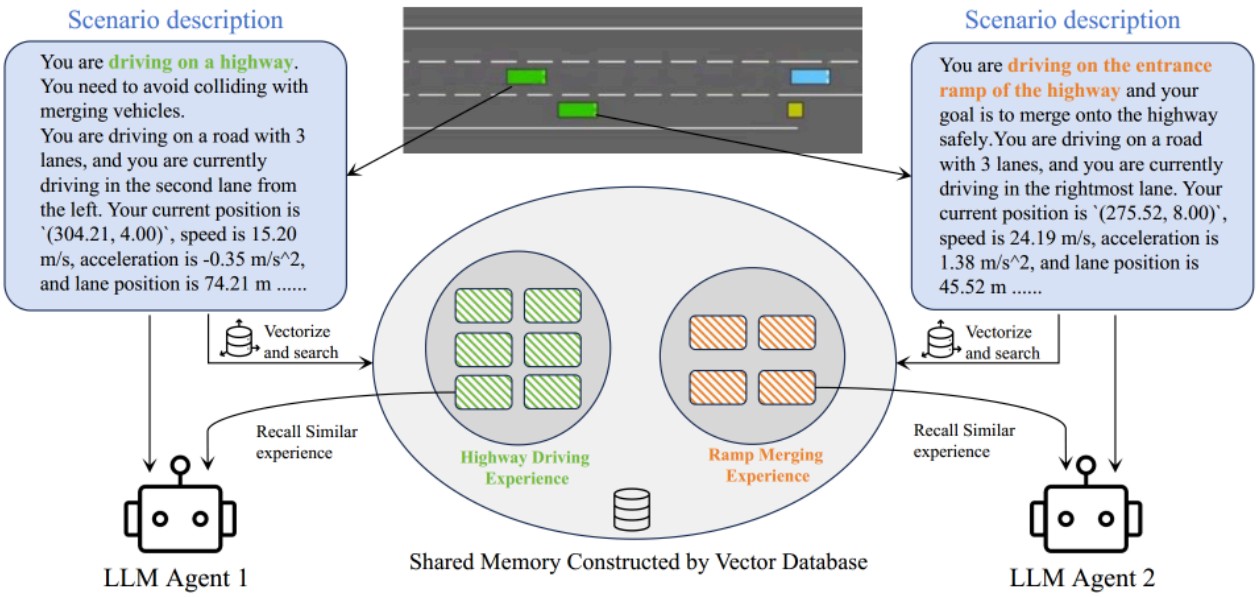


fig4.共享记忆模块

Ranking-based Reflection：做经验筛选

安全指标（TTC）：$Safety_{score}(t) = \begin{cases} \dfrac{20\,(TTC(t) - 1.5)}{3}, & \text{if } 1.5s < TTC(t) < 3s \\ 10, & \text{else if } TTC(t) > 3s \\ 0, & \text{otherwise} \end{cases}$

效率指标：通过车辆速度与周围平均速度比较

$$Efficiency_{score}(t) = \begin{cases} 10, & \text{if } V_{LLM}(t) > V_{Avg}(t) \\ 0, & \text{otherwise} \end{cases}$$

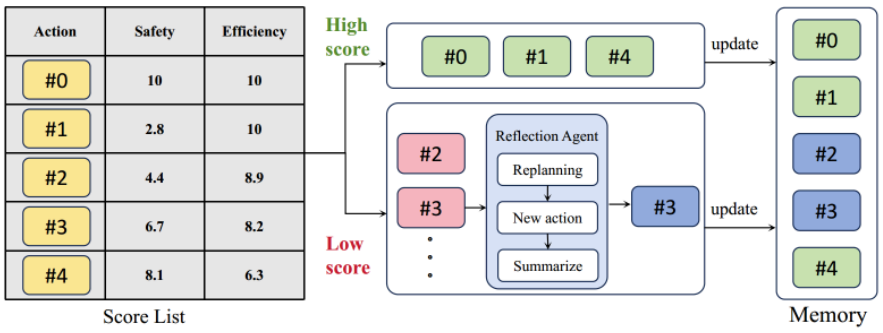


fig5.给每一步历史动作打分，筛出低质量决策，让 LLM 重新规划并生成更好的动作，再把修正后的经验写回共享记忆。

4、实验

仿真环境：highway-env （高速公路匝道汇入）

LLM： GPT-4

Text embedding model：text-embeddings-ada-002

Vector database：Chroma

Parameter	Value
The lane count of the highway main road	2
The lane count of the on-ramp merging road	1
The length of on-ramp merging road	120 m
The initialization speed of vehicles	range from 20 to 25 m/s
The spacing of the spawn points	40 m
The initial location noise	range from -10 to 10 m
The count of LLM controlled vehicles	2
The count of intelligent driver model vehicles	5
Policy frequency	2 Hz

table2.场景设置——双车道匝道汇入，半秒一次决策
当 agent 到达终点或与其他车辆发生碰撞时，视为场景结束

实验① KoMA vs MARL （multi-agent reinforcement learning）方法对比 fig7 成功率70%

实验② Shared Memory 消融 fig8

实验③ Muti-step 消融 fig9 table3

实验④ 泛化性 （改变车道数;从匝道迁移到环岛；其他 LLM 模型）fig10；fig11-12；

GPT-3.5、GPT-4、Llama3-8B、Llama2-7B、Qwen2-7B

5、总结

决策框架，感知来自仿真环境文本描述

单 Agent → Multi-Agent，两个 LLM 通过推断彼此行为意图实现交互（隐式），不涉及 LLM 参数训练

Action-only → Goal-Plan-Action

Private Memory → Shared Memory

Collision Reflection → Ranking-based Reflection

AGENTSCODRIVER: Large Language Model Empowered Collaborative Driving with Lifelong Learning

GPT-3.5-turbo（不涉及参数训练） + LangChain + HighwayEnv

1、背景/动机

可解释性、持续学习能力（经验检索）

2、总体框架

多车协同数学建模：分散式部分可观测马尔可夫决策过程 D-POMDP

Decentralized Partially Observable Markov Decision Process;

$\langle \mathcal{A}, S, \{A_i\}, \{O_i\}, Z, G \rangle$ agent, state, Action, Observation, 观测概率模型，共同goal；关键在于策略 $\pi_i : O_i^{1:t} \rightarrow A_i$ 的设计（本文方法）

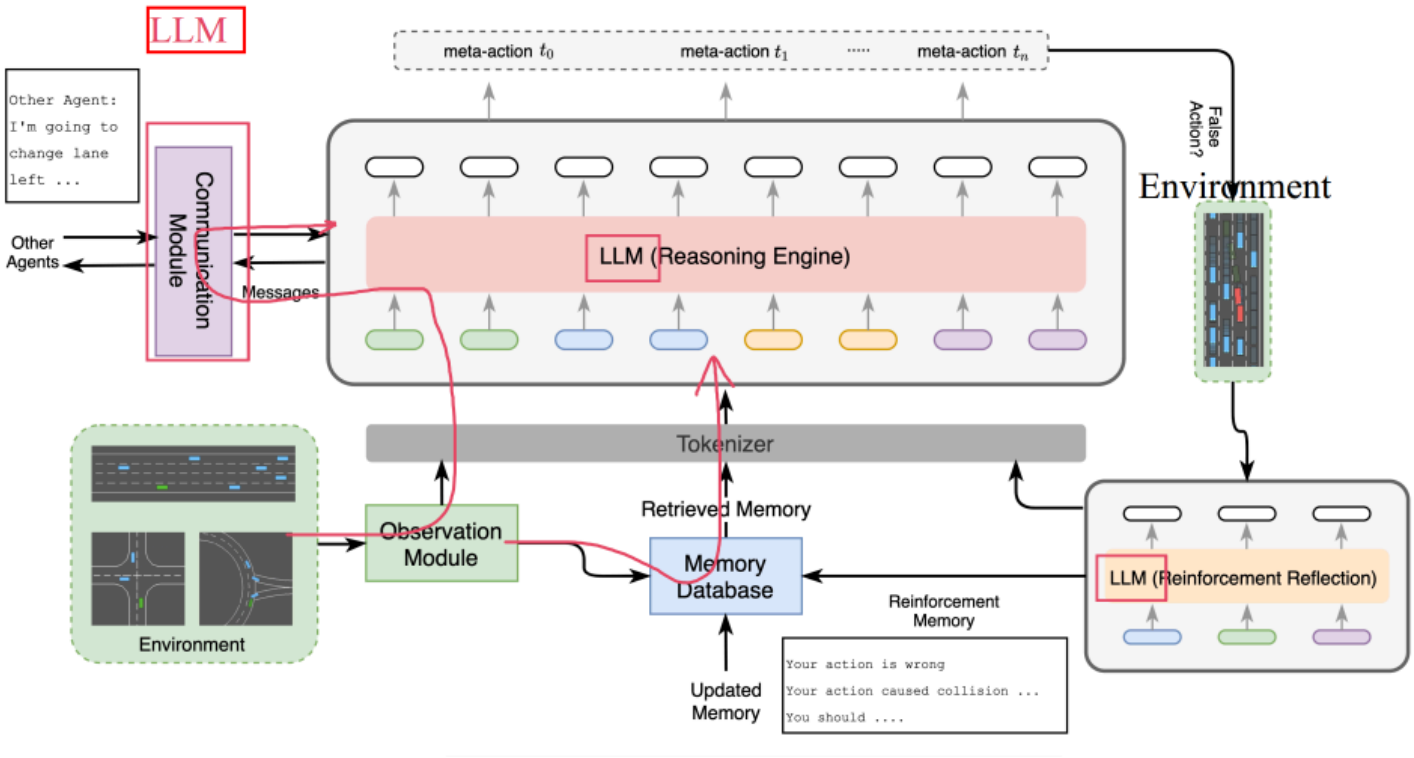


fig2.总体架构 five modules: observation module, reasoning engine, memory module, reinforcement reflection module, and communication module.

3、方法

五个模块

Observation：从 HighwayEnv 获得 text description

Cognitive Memory：Commonsense (rule) + Experience（retrieva) + Reflection

Reflection 包含：场景描述，有效推理过程，合理决策，以及吸取的经验教训

检索方法：Top-K

Reasoning Engine（LLM）：

Prompt：

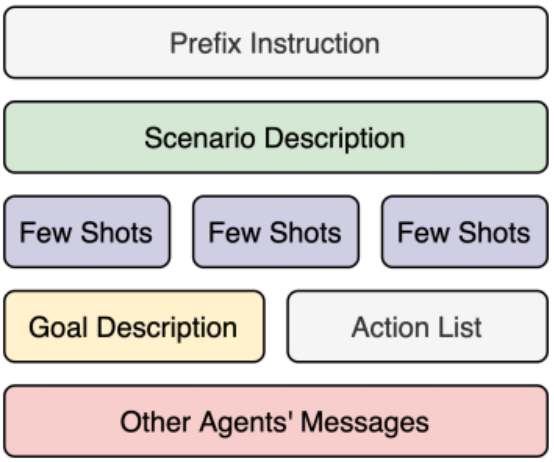


fig3.prompt 设计，few-shots 来自记忆模块

Reasoning Process: CoT 分解问题

Motion Planning：动作空间中选择

Reinforcement Reflection（LLM）： 强化 Agent 的经验记忆，通过反思提高后续决策能力

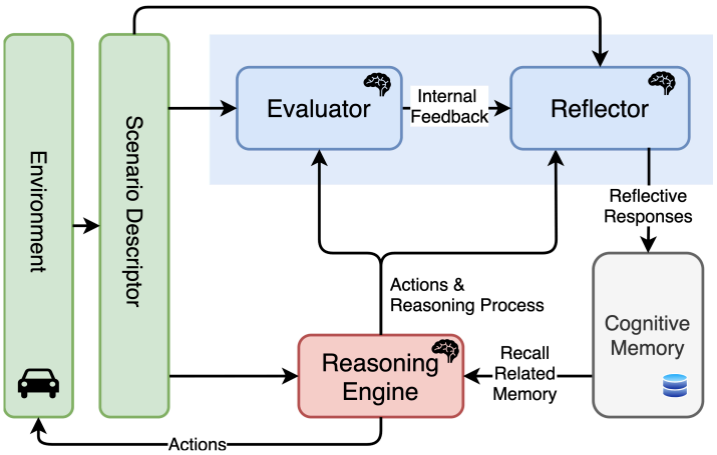


fig4.

LLM 两个角色：Evaluator 给出评分，Reflector 给出 错误分析和修正建议

Communication（LLM）： 扩大单车信息范围，允许车辆之间协商 intention

LLM 作为消息生成器 （Tool），当 Agent 认为需要通信时，自行调用

4、实验

仿真环境：HighwayEnv （Highway+Intersection）

应用框架：LangChain

LLM：GPT-3.5-turbo，不涉及 LLM 训练

指标：Success Step、Success Rate

Scenario	No. of agents	No. of shots	SS_{min}	SS_{Q1}	SS_{median}	SS_{Q3}	SS_{max}	SS_{mean}
Highway	2 agents	0 shot	2.0	2.0	4.0	7.0	12.0	4.2
		1 shot	2.0	3.0	5.0	6.0	14.0	5.2
		3 shots	4.0	5.0	8.0	17.0	30.0	11.0
		5 shots	4.0	6.0	8.0	20.0	30.0	13.5
	1 agents	0 shot	2.0	3.0	3.5	6.0	20.0	5.6
		1 shot	3.0	3.0	4.0	5.0	23.0	6.4
		3 shots	4.0	5.0	5.5	10.0	30.0	9.1
		5 shots	4.0	5.0	16.5	30.0	30.0	17.3
Intersection	2 agents	0 shot	4.0	6.0	6.5	10.0	20.0	8.2
		1 shot	5.0	6.0	7.5	9.0	30.0	10.4
		3 shots	6.0	8.0	12.0	21.0	30.0	15.6
		5 shots	4.0	7.0	15.0	30.0	30.0	17.6
	1 agents	0 shot	5.0	6.0	7.0	9.0	10.0	7.1
		1 shot	5.0	6.0	10.0	18.0	28.0	12.1
		3 shots	3.0	6.0	8.5	20.0	30.0	13.0
		5 shots	5.0	7.0	25.0	30.0	30.0	19.6

table2.主要结果—— 每个场景设置使用不同的初始化种子重复10次，以获得最终结果，包括SSmin、SSQ 1、SSmedian、SSQ 3、SSmax和SSmean；1/2 agent +15 simulator vechicle

实验① AgentCoDriver/Memory 有效性（Few-shot）：K↑，SSmean↑

实验② 对比实验 DiLu （Tab3)

实验③ Lifelong Learning：MemorySize↑ Performance↑ (Fig6)

实验④ 消融：Reflection、Communication

NO. of Agents	SR (w/o)	SR (w)	Growth Ratio
3	45.0%	75.0%	66.7%
5	40.0%	70.0%	75.0%

tab5.多agent 通信模块消融

局限：推理延迟（几秒）；单一模态（text）；场景简单（Sim2Real Gap）；Action space；Lifelong 依然是 data-driven

5、总结

AGENTSCODRIVER 是一个基于 GPT-3.5-turbo 的 muti-agent 协同驾驶框架，通过结构化环境描述、基于相似度检索的认知记忆、CoT 驾驶推理、Evaluator–Reflector 强化反思以及车辆间自然语言通信，实现多车协同决策与基于外部记忆积累的 lifelong learning；实验在 HighwayEnv 的 Highway 和 Intersection 场景中验证了 Memory、Reflection 和 Communication 对驾驶成功率及持续决策能力的提升。

CoMAL: Collaborative Multi-Agent Large Language Models for Mixed-Autonomy Traffic

CoMAL 面向混合自动驾驶交通的 **LLM 多智能体协同决策框架，**利用 **GPT-4o-mini**/Qwen 系列 LLM 作为多个 CAV Agent 的高层协作与推理模块，通过共享消息池实现多车通信（语言交流）、策略讨论和角色分配，再由各 Agent 生成 **IDM 参数**完成低层控制（传统车辆控制模型执行）。

1、背景/动机

LLM 增强自动驾驶系统主要优化自车驾驶性能，而混合自动驾驶交通需要研究整体交通效率（通过少量 CAV 改善整个交通流）。Mixed-Autonomy Traffic：connected autonomous vehicles（CAV）+ Human-driven

传统方法：基于规则、MARL——泛化能力和可解释性差

2、总体框架

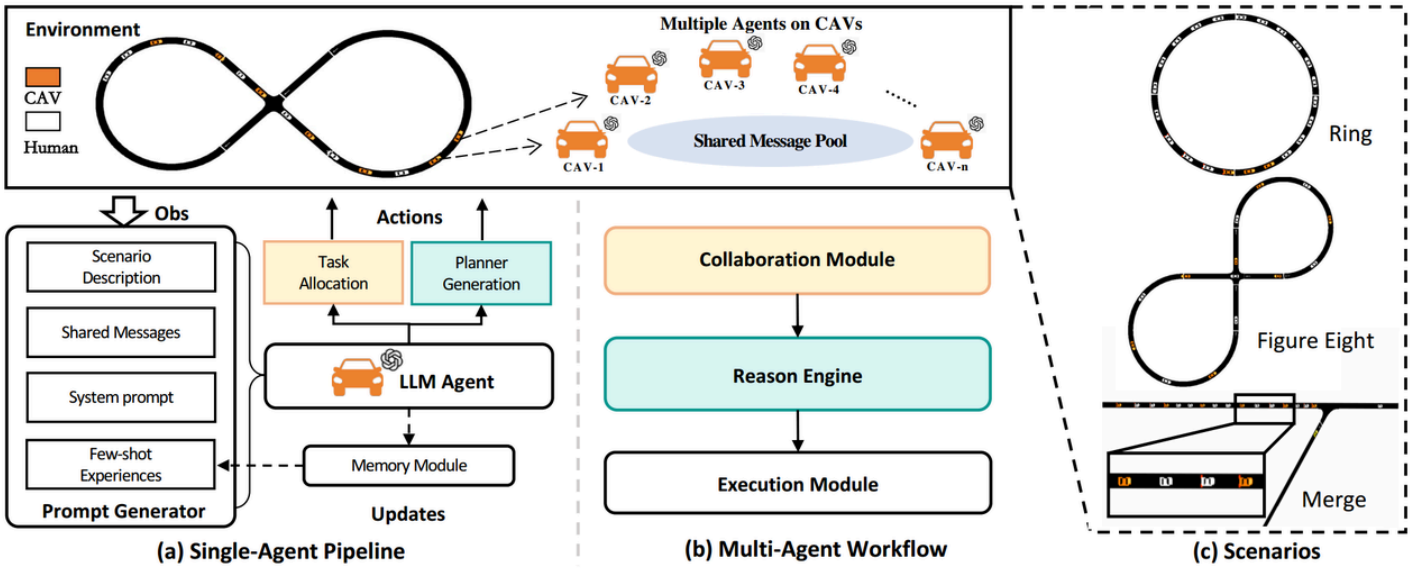


fig1.系统分两层：（a) 单车决策（b）多车协作

3、方法

单车：场景描述（text）、Memory（预定义 + LLM 更新）、Shared Message（接收其他agent信息）

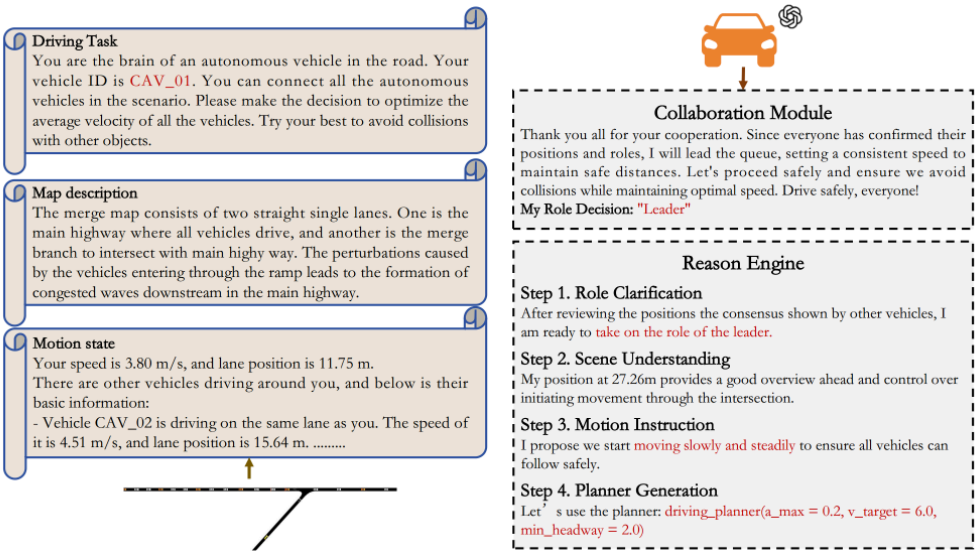


fig2.左侧：prompt示例 右侧合作和推理案例

多车：（three-stage）

① Collaboration

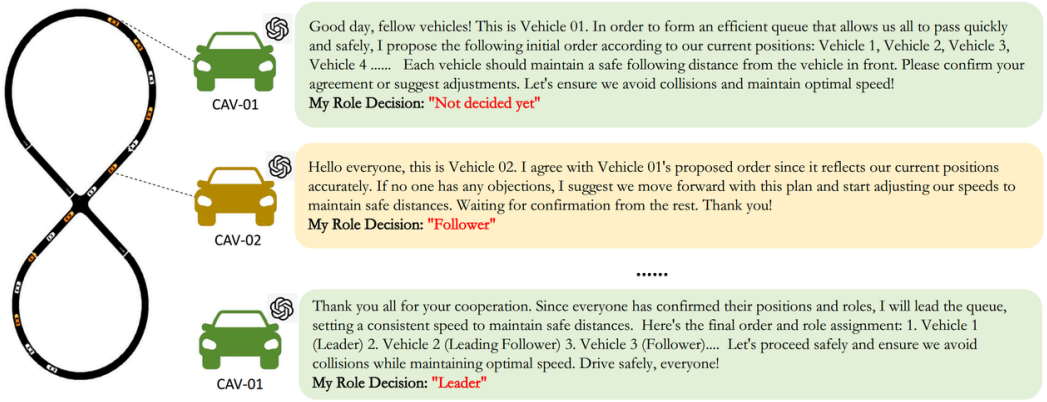


fig3. Agents 交互示例，在 Shared Message Pool 中发布和读取消息

② Reason Engine：输入Scenario description、Role、System prompt，输出IDM控制器参数 $(v_0, a_{\max}, s_0)$；CoT：fig2四步骤

③ Execution：LLM 做高层规划，IDM 做底层控制（Planner）

4、实验

仿真环境：Flow benchmark/ SUMO simulator

模型：GPT-4o-mini / Qwen72B/32B/7B

三个实验场景 fig1/tab1参数：Ring（环形道路）、Figure Eight（八字环路）、Merge（匝道汇入）

两个指标：平均速度（↑，交通效率↑）、速度标准差（↓，稳定↑）

① 主结果（tab2）CoMAL vs Human Driver

Table 2: Quantitative Evaluation of CoMAL on Flow Benchmarks

Metric	Model	FE 0	FE 1	FE 2	Ring 0	Ring 1	Ring 2	Merge 0	Merge 1	Merge 2	Merge 3	Merge 4
Avg	Human Driver	5.61	5.61	5.61	**2.88**	**2.88**	**2.88**	6.40	6.40	6.40	6.40	6.40
	CoMAL	**6.40**	**6.47**	**6.29**	2.86	2.85	2.87	**6.59**	**7.40**	**7.42**	**7.86**	**8.83**
Std	Human Driver	4.55	4.55	4.55	0.79	0.79	0.79	3.12	3.12	3.12	3.12	3.12
	CoMAL	**1.74**	**1.77**	**2.24**	**0.29**	**0.26**	**0.31**	**2.88**	**2.91**	**2.61**	**2.47**	**2.70**

② 消融实验：Perception、Memory、Collaboration （合作模块：避免所有车辆采取类似策略而产生冲突）

③ LLM 模型比较：GPT-4o-mini 最优

④ 与 RL 方法比较：FE中CoMAL更优，Merge中RL更优（Merge需要更强局部控制）

DAPlanner: Dual-agent framework with multi-modal large language model for autonomous driving motion planning

双 Agent 结构：生成 （Vicuna-v1.5-7B）+ 修正（LLaVA-v1.5-7B）

Scene Analysis → Behavior Decision → Trajectory Generation

Issue Analysis → Optimization Strategy → Trajectory Update

1、背景/动机

研究方向：运动规划——综合其他交通参与者、道路条件和驾驶任务，从当前状态设计到目标状态的合理轨迹

现有方法：

Rule-based Planning

Learning-based Planning（深度学习）：泛化和可解释性差、信息融合困难

LLM-based Planning：

道路拓扑表示困难（地图表达不足）

自动驾驶输入复杂（全文本会导致token数量大，空间关系丢失）→ 空间信息放 BEV，关键数值信息放文本（多模态输入）

LLM 直接生成轨迹不可靠 → 再加入一个Agent 做检查修正

2、总体框架

问题描述：$\mathcal{T}raj = \mathrm{DAP}(\mathcal{O}bstacle, \mathcal{M}ap, \mathcal{E}go)$

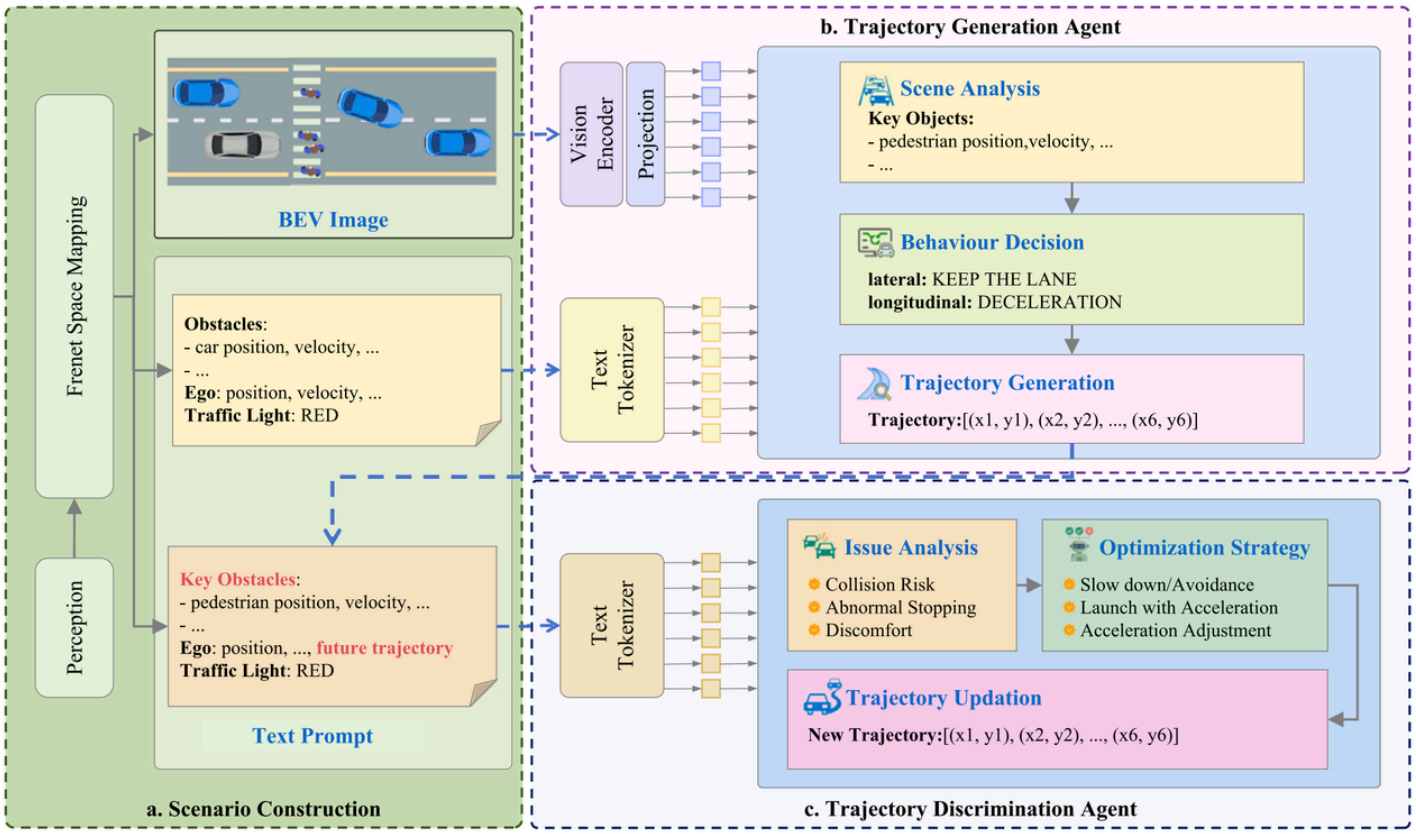


fig2.总体框架（三模块）：场景构建、轨迹生成Agent、轨迹判别Agent

Agent	模型	输入	功
Trajectory Generation Agent	Vicuna-v1.5-7B 为语言核心 + 视觉编码	BEV + text	Scene Analysis → Behavior D Generation
Trajectory Discrimination Agent	LLaVA-v1.5-7B	Scene + 初始轨迹	Issue Analysis → Optimizatio

3、方法

a.Scenario Construction

笛卡尔坐标 → **Frenet 坐标**：把不同弯曲道路上的二维运动，统一转换成相似的“沿路前进s + 横向偏移l”表示，从而减少道路结构对学习过程的干扰，也能够更自然地表征车路及参与者的相对关系。

BEV visual information（Frenet 空间中的各种地图元素）+ text numerical information（物体坐标、速度、交通灯状态）

b.Traj generation agent

Step1. Feature Encoding —— CLIP-ViT-L-336px $I \xrightarrow{\text{CLIP}} F_v \xrightarrow{\text{Projection}} F_v'$，视觉编码+对齐语义空间

Step2. Scene Analysis —— key object

Step3. Behavior Decision —— 横向行为+纵向行为

Step4. Traj. Generation —— 在Frenet空间的轨迹，然后后处理回笛卡尔坐标

c.Traj. Discrimination Agent

独立训练的Agent，会分析异常原因并提出具体改进策略

Step1. 问题分析：检查 Collision + Abnormal Stop + Discomfort

Step2. 优化策略（基于异常类型分析

Step3. 轨迹更新

数据集构建：

根据两个agent各自任务构建两套数据

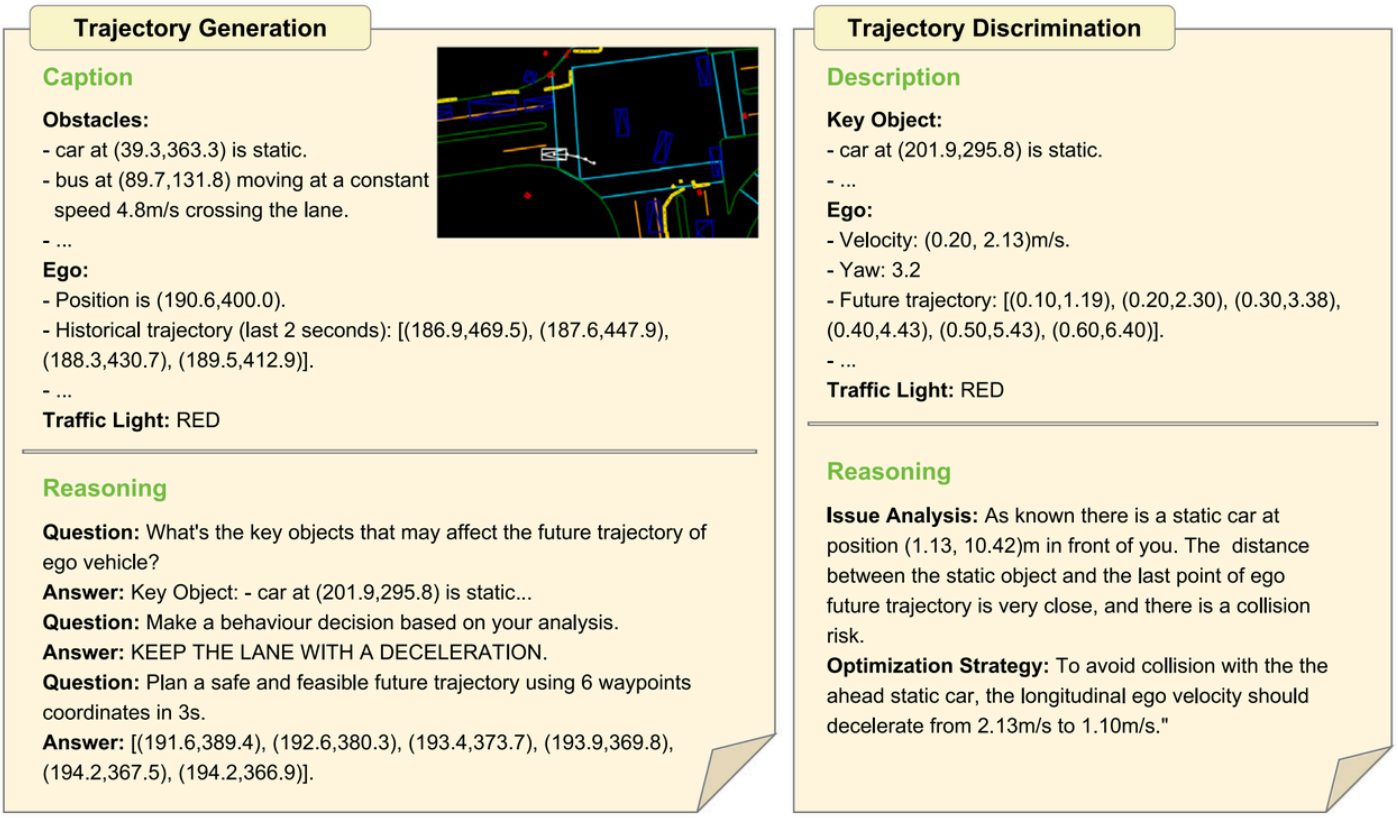


fig3. 示例

Generation：nuScenes 过去2s+未来3s，BEV中白色框表示自我车辆的位置、方向和形状，与颜色匹配的虚线表示这些对象的历史轨迹点。橙子线表示车道分隔线，绿色线表示道路边界，而青色线表示行人过路处。

Discrimination：人工制造异常数据和文本描述，标注异常和优化建议

4、实验

数据集：nuScenes （测试validation set）

模型：Generation Agent：vicuna-v1.5-7B ；Discrimination Agent：LLaVA-v1.5-7B

指标：L2、碰撞率（ST-P3协议）；道路边界交叉率（与车道边界的碰撞风险，即驶出道路的危险情况）

结果：

① 主结果

Table 1
Motion planning performance. The top and runner-up performances are respectively indicated by bold and underline formatting.

Method	L2 (m)				Collision (%)				Intersection (%)			
	1 s	2 s	3 s	Avg.	1 s	2 s	3 s	Avg.	1 s	2 s	3 s	Avg.
NMP [57]	–	–	2.31	–	–	–	1.92	–	–	–	–	
SA-NMP [57]	–	–	2.05	–	–	–	1.59	–	–	–	–	
FF [58]	0.55	1.20	2.54	1.43	0.06	0.17	0.07	0.43	–	–	–	–
EO [59]	0.67	1.36	2.78	1.60	0.04	0.09	0.88	0.33	–	–	–	–
ST-P3 [37]	1.33	2.11	2.90	2.11	0.23	0.62	1.27	0.71	2.53	8.17	14.4	8.37
UniAD [38]	0.44	0.67	**0.96**	0.69	0.04	0.08	0.23	0.12	**0.21**	1.32	3.63	1.72
VAD-Tiny [39]	0.46	0.76	1.12	0.78	0.21	0.35	0.58	0.38	0.94	3.22	7.65	3.94
VAD-Base [39]	0.41	0.70	1.05	0.72	0.07	0.17	0.41	0.22	0.60	2.38	5.18	2.72
Drive-WM [41]	0.43	0.77	1.20	0.80	0.10	0.21	0.48	0.26	–	–	–	–
Ours	**0.33**	**0.63**	1.03	**0.66**	**0.01**	**0.05**	**0.19**	**0.08**	0.57	**0.88**	**1.07**	**0.84**

Tab1.规划表现（L2 最低并不等于唯一正确规划），安全性增益明显

② 消融实验：

Table 2
The influence of using different module combination on the metrics of average L2, Collision and Intersection in the 1–3 s period.

Trajectory Generation Agent	Frenet Space Mapping	Trajectory Discrimination Agent	Data Augmentation	L2 (m)	Collision (%)	Intersection (%)
✓				0.88	0.36	3.17
✓	✓			0.62	0.16	0.84
✓	✓	✓		0.67	0.10	0.84
✓	✓	✓	✓	0.66	0.08	0.84

Tab2.

Frenet主要解决道路几何约束

Discriminator 有时会把原本接近 GT 的轨迹修改成另一条安全轨迹（不是做轨迹拟合，而是提高安全性），因而L2↑但碰撞率↓

Data Augementation：异常轨迹训练数据（增加）对Discrimination agent有效

③ 不同LLM组合

④ 训练迭代与学习率

⑤ 定性分析

5、总结

强依赖高精地图 HD map；

只在nuScenes上做开环实验；

双大模型实时性；

DAPlanner 是一种基于双大模型 Agent 的自动驾驶运动规划框架，它首先将 HD Map、自车和交通参与者映射到 Frenet 空间，并构建 BEV 图像与文本形式的多模态场景表示；随后由基于 Vicuna-v1.5-

7B 的 Trajectory Generation Agent 按“场景分析—行为决策—轨迹生成”的推理链生成未来 3 s 轨迹，再由基于 LLaVA-v1.5-7B 的 Trajectory Discrimination Agent 对碰撞、异常停车和舒适性问题进行反思分析并修改轨迹，以提高最终规划的安全性。